Introducing Logic Transformer

King-Yin Yan[0009−0007−8238−2442]

general.intelligence@gmail.com

Abstract. Logic Transformer is a differentiable neural network that explicitly performs classical symbolic logic inference (with rules matching, substitutions, etc). Preliminary experiments show that it achieves a very high compression ratio (ie fewer parameters) and also seems to train faster as a consequence. From the "No Free Lunch" perspective this suggests that algebraic/categorical structure of logic is suitable for compressing our world (as Wittgenstein says, logic is a mirror-image of the world). A high compression ratio forces the knowledge contained in our neural network to attain "logic coherence", which we believe is the antidote to hallucinations. The explicit logic structure also opens the "black boxes" of neural networks for inspection and interpretation. Moreover, it allows to use a classical symbolic logic inference engine (eg. OpenCog, OpenNARS) side-by-side with our neural network in a hybrid architecture, that may offer advantages we are only beginning to explore. For all these implications, we claim that this is a key innovation that solves the AGI problem.

Keywords: logic-based AI · neural-symbolic integration · LLMs (large language models)

1 Background: RL + LLM Architecture

This section outlines the basic framework required for subsequent work. We aim to combine RL (reinforcement learning) and LLMs (fig. 1). LLMs are traditionally implemented by Transformers, to be replaced by Logic Transformers.

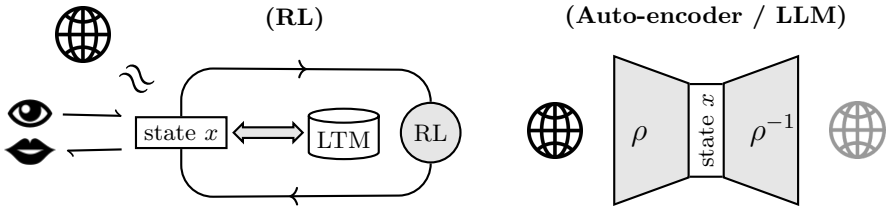


Fig. 1. Left: basic RL setup; LTM = long term memory. Right: The essence of an auto-encoder, of which LLMs are a special case, is to compress (via ρ) and reconstruct (via ρ^{-1}) world data.

Two types of architectures are possible (fig. 1), where a traditional LLM may serve as a world-model (**W**) or as a model of thinking processes (**T**). It turns out the latter is more natural and similar to mainstream approaches such as RLHF [11]. It implies that "prompts" are regarded as Working Memory and offers the tremendous advantage that humans can read the thoughts of an AGI directly in natural language.

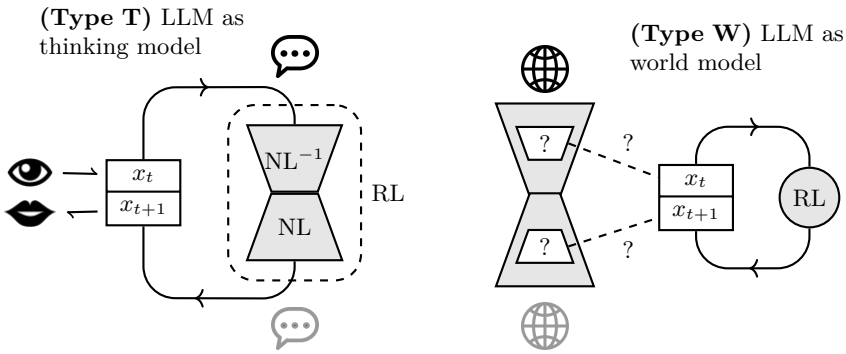


Fig. 2. Two types of RL + LLM architectures. NL^{-1} is a map that compresses natural language to a hidden representation (not shown explicitly).

2 Logic Transformer

In classical logic-based AI, a single step of inference is a mapping $\Gamma \overset{\text{KB}}{\longmapsto} \Gamma + \delta$ where KB is a set of logic **rules** and Γ is a set of ground propositions or **facts**. Γ also called the state $\boldsymbol{x}$ or Working Memory. "Set" implies algebraic structure, ie, permutation invariance of elements. We recognized early on that this has a connection with the **equivariant** structure of Self-Attention; but it was not obvious how to extract further algebraic structures from logic and to apply such symmetries; instead, we found a way to actually implement the syntactic operations of substitutions and logic-rule matching, so the resulting neural network obviously inherits logic structure implicitly. A logic rule is a formula with variables such as:

$$\forall X\ \forall Y\ \forall Z.\quad \mathrm{father}(X, Y) \wedge \mathrm{father}(Y, Z) \Rightarrow \mathrm{grandfather}(X, Z) \tag{1}$$

where the variables must obey strict discipline (X matches to X, etc) as shown by dotted lines.

A key innovation is what I termed the **cylindrification factor** γ, which allows a logic constant to change continuously into a variable, as illustrated in fig. 3.

Furthering the idea, the matching (or **unification**) of variables makes use of a fixed number of variable "slots" where we "move" contents in and out of the

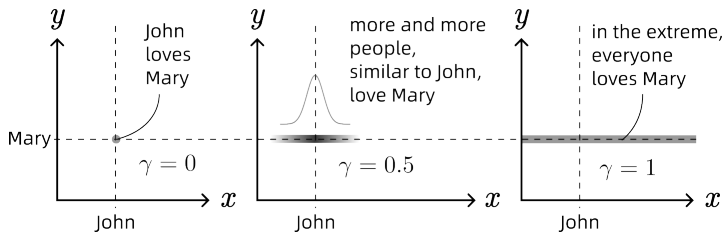
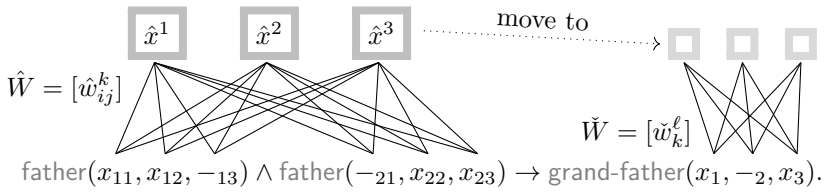


Fig. 3. Illustration of the cylindrification factor γ.

slots by neural weights with **Softmax**, so that the transfer of contents can vary continuously:

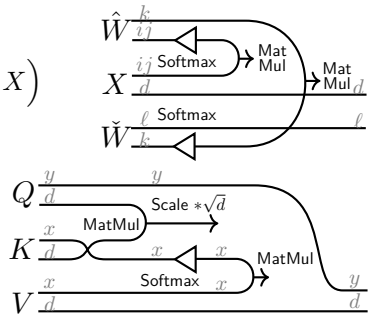


(2)

Interestingly, substitution / unification has a form very similar to **Self-Attention** except for the order of softmax and tensor product (with neural string-diagrams explained in the next section):

$$Z := \oslash \check{W} \otimes \left(\oslash \hat{W} \otimes X \right)$$

$$\alpha := \left(\oslash \frac{\langle Q, K \rangle}{\sqrt{d_k}} \right) V$$



(3)

The next step would be to match WMEs (Working Memory Elements, ie, propositions) to rule arguments. This is the point when we stopped and submitted our last paper, which was rejected. We concluded that rules matching was too complicated and not worth the trouble. It turns out what was needed is the permutation of all WMEs, which is simply the Cartesian self-product $X^{\times n}$. This looks computationally prohibitive, but we're just back to the old problem faced by the Rete algorithm [4], and there are many new (differentiable) ways to solve this problem. An initial version of our system can solve TicTacToe quite efficiently. Further testing confirmed that it can solve natural-language tasks such as bAbI [15] and TinyStories [3] with 10x fewer parameters than traditional Transformers (Fig 5L). We attribute this efficiency to the logic structure inherent in the Logic Transformer neural network.

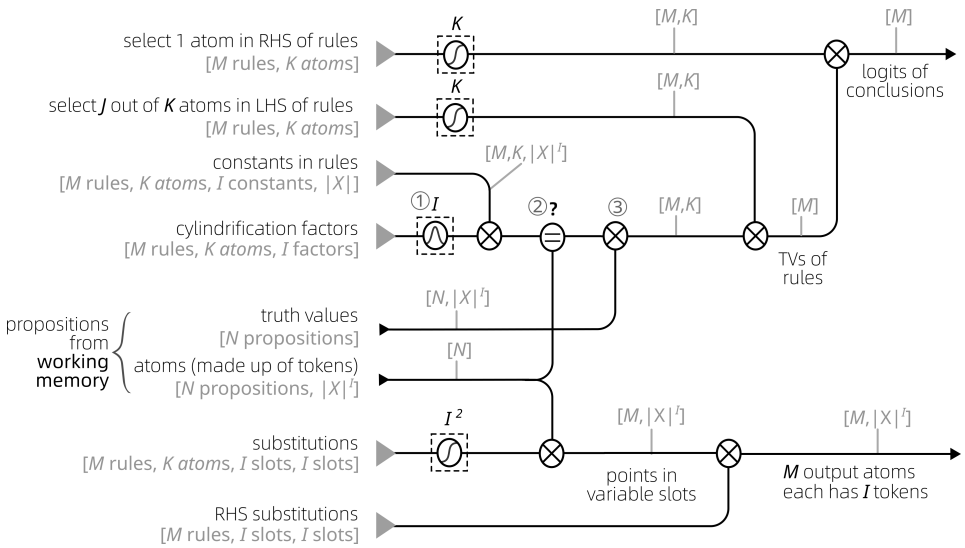


Fig. 4. String-diagram of the Logic Transformer.

2.1 Source code and experiments

We went through 3 stages of experiments. All source code is under Creative Commons Zero-1.0 license and can be accessed from:
`http://github.com/Cybernetic1/`{insert name}

1	TicTacToe experiments with traditional neural nets and Transformers	`RL-TTT-experiments`
2	TicTacToe experiments with Logic Transformer	`logic-autoencoder-TTT`
3	AGI experiments with bAbI and Tiny Stories	`AGI-experiments`

Readers can use Copilot to ask questions about our code and even contribute new material. The repository is experimental code and contains >80 Markdown documents, mostly generated by Copilot, which may be difficult for beginners to navigate. Feel free to contact the author for assistance.

2.2 On the algebraic structure of logic and string diagrams

A natural question arises whether it is possible to simplify the string diagram (Fig. 4) to make it computationally more efficient. Some prior research has been done in this direction. The diagrams in equation (3) are **Functor String Diagrams** as described in [1]. They have the advantage of keeping track of dimensions of tensors as well as preserving equivariant structure – these seem to be all we need for implementing symbolic logic.

Let's digress a little into what we mean by "logic structure." The modern approach is via categorical logic [8] [10], where logic structure is encapsulated

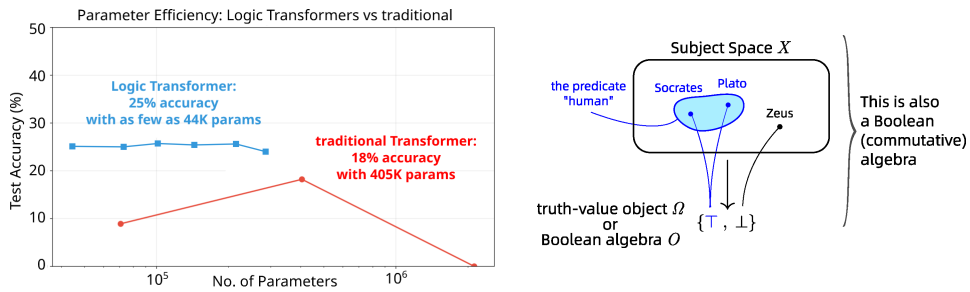


Fig. 5. (Left) Compression ratio, Logic Transformer vs traditional. (Right) Paul Halmos' lifting of propositional logic to predicate logic.

as CCCs (**Cartesian closed categories**) or as **topoi** [7]. The categorical approach has Curry-Howard correspondence [12] at its core. A parallel approach is based on relation algebra and leads to the notion of **allegories** [5]; it is inter-translatable with categorical logic. Paul Halmos developed an early idea of algebraic logic based on lifting propositional logic to predicate logic via a morphism that preserves Boolean algebraic structure, as illustrated in Fig. 5R. This idea is later developed into the notion of **hyperdoctrines** by Lawvere.

In the study of neural-symbolic integration, a central concern is the equivariant property $A \wedge B = B \wedge A$, which we believe is also a key feature of Self-Attention [14] and thus Transformers. The categorical way to capture permutation invariance is by **symmetric monoidal categories** (SMCs), which has the visual language of **string diagrams**. **Compact closed** SMCs have dual objects and cup $\cup$ and cap $\cap$ morphisms. These allows to deal with operations such as tensor contraction (eg. dot product) – required by the Attention mechanism. We can **re-write** such string diagrams while preserving their symmetric / algebraic properties, eg [2]. This sounds like a very exciting research direction, though unfortunately because the space of continuous functions has uncountable basis, the completeness of such rewriting cannot be guaranteed. What we have is equational completeness in the sense of $[\![D_1]\!] = [\![D_2]\!] \Rightarrow D_1 \overset{*}{\leftrightarrow} D_2$.

There is a path from SMCs $\rightarrow$ compact closed SMCs $\rightarrow$ Cartesian closed categories $\rightarrow$ topos, with increasingly more structure. We know how to construct neural networks in SMCs so the challenge is how to construct neural networks in toposes. There is some research in this direction [13] [6], which may shed light to our approach.

The string diagram in Fig. 4 is of an older style, as in [9]. The model is impractical as it assumed the naive Cartesian product X^n of Working Memory. We are working with some modified version of it, but the claims of this paper remains basically truthful.

3 Additional Features and Enhancements

If the reader is familiar with classical logic-based AI (or the "neo-classical" OpenCog or OpenNARS), and if you understand our main innovation, the Logic Transformer, as a differentiable "rules-base," then you would have no problem seeing many natural developments that follow from it. If we were to explain them in detail, this paper would blow up to book-length. Instead we just list some new features briefly:

Vector quantization (VQ) of propositions	Seems to be a nice trick, introduced by Copilot without the author's permission. It is a well-known problem in RL that the action space is often too large to be computationally tractable; in the AGI case that action space is the space of all possible / imaginable propositions. Copilot claims that they can be compressed into 8K propositions using a VQ code book. When asked, it explains that it is not 8K distinct propositions, but propositional patterns (see doc #61). Examples like this make us suspect that Copilot is already capable of developing AGI with just a modicum of human guidance.
Hybrid architecture with symbolic logic engine	As our differentiable logic rules become cleanly separable entities, the neural network can easily co-operate with a traditional symbolic inference engine. With a layer of adaptation, it can work with OpenCog/NARS. What advantages will it bring? One example is the ability to quickly incorporate new knowledge into the thinking process. For example, when told that for an Abelian group, $a \cdot b = b \cdot a$, an LLM first places this fact into Working Memory (ie. the prompt), and this may cause the algebraic rule to be applied to new inputs, but this happens through an opaque process, known as long-range dependency, in traditional Transformers. With a symbolic logic engine, the fact $a \cdot b = b \cdot a$ can be directly translated into a logic rule and be immediately applicable.
Representation of Working Memory	One of the key design choices of an AGI how to represent Working Memory. OpenCog proposes a hypergraph structure. We explored this issue at length with Copilot and Qwen, particularly on trees because Chomsky revealed them to be a universal underlying structure of natural language. We looked at continuous representations of trees, whose algebraic structure is captured by **F-algebras**. Even so, faithfulness is very hard to guarantee in finite dimensional space. Many differentiable tree-rewriting techniques are complicated to implement. We concluded that it is probably more efficient to use propositional logic rules for rewriting a graph-based Working Memory.
neo-Davidsonian grammar	A logical form in linguistics, based on events. For example loves(John,Mary) would be loves(e) $\wedge$ Agent(e,John) $\wedge$ Patient(e,Mary) where e is an event. This technique was also suggested by Copilot and adopted by the author.
Reflection	Refers to inference traces being made into inspectable objects fed into Working Memory. This allows an AI agent to examine its own thought processes. Some researchers regard this a core feature of "consciousness."

HOL	Higher-Order Logic is needed to learn highly abstract rules that may achieve better compression.
Horn form	A logic rule of the form $X \wedge Y \wedge W \wedge ... \rightarrow Z$. Sort of a relic from the past, it makes the implementation of classical symbolic engines easier, eg. PROLOG. Some discussions with Copilot suggest that it is no longer needed; a rule can have multiple heads.

References

[1] Vincent Abbott. *Neural Circuit Diagrams: Robust Diagrams for the Communication, Implementation, and Analysis of Deep Learning Architectures*. 2024. arXiv: 2402.05424 [cs.LG]. URL: https://arxiv.org/abs/2402.05424.

[2] Filippo Bonchi et al. "String Diagram Rewrite Theory I: Rewriting with Frobenius Structure". In: *J. ACM* 69.2 (Mar. 2022). ISSN: 0004-5411. DOI: 10.1145/3502719. URL: https://doi.org/10.1145/3502719.

[3] Ronen Eldan and Mark Li. "TinyStories: How Small Can Language Models Be and Still Speak Coherent English?" In: (2023). URL: preprint%20arXiv:2305.07759.

[4] Charles Forgy. "Rete: A Fast Algorithm for the Many Patterns/Many Objects Match Problem". In: *Artif. Intell.* 19 (1982), pp. 17–37. URL: https://api.semanticscholar.org/CorpusID:62572191.

[5] P.J. Freyd and A. Scedrov. *Categories, Allegories*. North-Holland Mathematical Library. North Holland, 1990. ISBN: 9780080887012. URL: https://books.google.com.hk/books?id=fCSJRegkKdoC.

[6] Bruno Gavranovi et al. *Position: Categorical Deep Learning is an Algebraic Theory of All Architectures*. 2024. arXiv: 2402.15332 [cs.LG]. URL: https://arxiv.org/abs/2402.15332.

[7] Goldblatt. *Topoi – the categorical analysis of logic*. 1984, 2006.

[8] Bart Jacobs. *Categorical logic and type theory*. Elsevier, 1999.

[9] Nikhil Khatri et al. "On the Anatomy of Attention". In: (2024). arXiv: 2407.02423 [cs.LG]. URL: https://arxiv.org/abs/2407.02423.

[10] Lambek and Scott. *Introduction to higher order categorical logic*. Cambridge, 1986.

[11] Zihao Li, Zhuoran Yang, and Mengdi Wang. *Reinforcement Learning with Human Feedback: Learning Dynamic Choices via Pessimism*. 2023. arXiv: 2305.18438 [cs.LG]. URL: https://arxiv.org/abs/2305.18438.

[12] Sørensen and Urzyczyn. *Lectures on the Curry-Howard isomorphism*. Elsevier, 2006.

[13] David I. Spivak. *Category Theory for the Sciences*. Cambridge, Massachusetts: MIT Press, 2014.

[14] Vaswani et al. "Attention is all you need". In: (2017). https://arxiv.org/abs/1706.03762.

[15] Weston, Chopra, and Bordes. "Memory networks". In: *ICLR (also arXiv)* (2015).